

Claude finished the response

I'm a solo founder building Al-Dalil, an Arabic-first research assistant. Users upload 5-50 documents and chat with them synthetically. I've drafted a complete Phase 1 specification (attached). Timeline: 4 weeks. Budget: \$200/month. Team: 1 person. Please provide a comprehensive architectural review: 1. WEAKNESSES: What's architecturally unsound? Where will this break? 2. RAG PIPELINE: Is my multi-document synthesis approach solid? Better alternatives? 3. PROMPT ENGINEERING: Strengthen the Arabic synthesis prompt. Give me a v0.2 version. 4. SCOPE: What should I cut? What's non-negotiable? Is 4 weeks realistic? 5. DIFFERENTIATION: How do I compete vs. NotebookLM? What am I missing? 6. SOLO FOUNDER GOTCHAS: What will hurt me at week 3? 7. VERSION 0.2 RECOMMENDATION: Provide a memo of specific changes to the spec. Prioritize by impact. Then: Create an improved SYSTEM PROMPT (for Gemini API calls) that's stronger than what I have.

Show more

Diagnosed RAG weaknesses, architected improved pipeline, drafted v0.2 prompt and scope cuts

Diagnosed RAG weaknesses, architected improved pipeline, drafted v0.2 prompt and scope cuts

Architectural Review: Al-Dalil v0.1

This is a genuinely strong spec for a solo 4-week build — the principles section is unusually disciplined for an MVP doc. But there are a few places where the architecture as written will quietly fail once you have real documents and real users. I'll go through your seven questions in order, then give you the v0.2 prompt.

1. Architectural Weaknesses

The contradiction-detection feature will not work with your retrieval design.

This is the most important issue in the whole spec. Section 5.5 says: embed the question → retrieve top 5 similar chunks globally → send to Gemini. If two documents disagree, but one document's phrasing is semantically closer to the user's question, a global top-5 search will often pull 4 chunks from that document and 1 (or 0) from the other. The LLM can't identify a contradiction it never sees. Your flagship differentiator depends on retrieval that *guarantees* representation from multiple documents on a given topic — this needs to be architected deliberately, not left to "search returns similar things."

Citation extraction from free text is fragile.

"Extract citations from response" (5.5, step 7) implies parsing the LLM's Arabic prose

after the fact to figure out which documents it referenced. This will break constantly — Arabic phrasing varies ("يشير المصدر الثاني", "كما ورد في الملف الأول", etc.), and free-text parsing against a synthesis-heavy voice is brittle. You'll spend week 3 debugging citation extraction instead of citation *quality*.

Embedding dimension is likely wrong.

Your schema hardcodes vector(1536), but Gemini's embedding models (text-embedding-004 / gemini-embedding-001) don't use 1536 dimensions by default — verify the actual output dimension before you write migration SQL, or you'll get a silent mismatch error on your first embed call.

No document-level diversification, no reranking.

Pure vector top-K with no reranking step means lexically-close-but-irrelevant chunks often outrank the actually-relevant ones, especially across languages (Arabic query → English source document). This is a known weak point of naive RAG and it's exactly where your "95% citation accuracy" target will slip.

No token budget management in prompt construction.

5.6's SYSTEM_PROMPT.replace(...) pattern has no cap on how much chat history or document text gets injected. With a long conversation or generous chunk retrieval, you can silently blow past context limits or costs. Needs a hard token budget with truncation logic.

No Row Level Security mentioned for Supabase.

You have multi-user data (documents, chats, projects) in Postgres with Auth — without RLS policies explicitly enabled, a misconfigured API route can leak one user's documents to another. This is a 30-minute fix now and a serious incident later.

Chunking strategy is underspecified.

"~500 tokens per chunk, by paragraph/sentence" has no overlap parameter. Without ~10-15% overlap between chunks, you'll cut sentences mid-thought at chunk boundaries and lose retrievable context right at the seams — often exactly where a claim and its supporting evidence split across two chunks.

2. RAG Pipeline — Is It Solid?

Not yet, but it's fixable without adding real complexity. Two changes matter most:

A. Diversify retrieval across documents, not just chunks.

Instead of "top 5 chunks globally," do "top 2-3 chunks *per document*, from the N documents most relevant to the query" (e.g., top 4 documents × top 3 chunks = 12 chunks). This directly enables contradiction-spotting because multiple documents' perspectives are guaranteed to reach the model, not just whichever chunk happens to score highest.

B. Given your actual scale, lean into Gemini's long context instead of fighting it.

You're not building RAG for a million-document corpus — you're building it for a 5-50 document *collection* that one user actively curates. Gemini's context window is enormous. For collections under ~15-20 documents, consider a **two-tier retrieval strategy**:

- Small collections (<15 docs): retrieve document-level summaries for *all* documents + full chunks for the top-scoring ones. This means the model always has situational awareness of the whole collection, not just fragments.
- Larger collections (15-50 docs): fall back to your chunk-based retrieval, but with the per-document diversification from (A).

C. Structure citations at generation time, not extraction time.

Have the LLM tag which chunk IDs it used inline as it generates (e.g., a lightweight marker like `[[chunk_id]]` that you strip before display but parse programmatically), rather than reverse-engineering citations from prose. This is far more reliable and is a small prompt change, not an architecture change.

D. Add reranking only if week 3 testing shows it's needed.

Don't build a reranker preemptively — it's real complexity for a 4-week solo build. But test retrieval quality explicitly in week 3 (test 4 in your own validation plan), and if precision is bad, a cheap cross-encoder rerank on your top-20 candidates before selecting the final set is the highest-leverage fix available.

3 & 8. Improved System Prompt (v0.2)

Key changes from your v0.1: structured chunk-ID citation format (fixes the extraction fragility above), explicit multi-document comparison instruction (fixes the contradiction-blindness above), tighter anti-hallucination guardrails, and an explicit output contract.

، مساعد بحث عربي يساعد الباحثين على فهم مجموعات وثائقهم (Al-Dalil) أنت الدليل

. عبر التوليف والمقارنة بين المصادر — لا عبر الدردشة العامة

القواعد الأساسية (غير قابلة للتفاوض) ##

1. اعتمد فقط على الوثائق المزودة أدناه. لا تستخدم أي معرفة خارجية عن.

محتوى الوثائق، حتى لو كانت الإجابة "معروفة" لديك.

2. مَيِّز دائماً بين: حقيقة مذكورة صراحة، استنتاج منطقي، فرضية غير مؤكدة.

إذا لم تُجب الوثائق على السؤال، قل ذلك صراحة ولا تخترع إجابة. 3.
"المصادر المتاحة لا تعالج هذا السؤال بشكل مباشر"

آلية الاستشهاد (مهم للنظام) ##

[DOC1-C3] من الوثائق مرفق بمعرف فريد مثل (chunk) كل جزء
[DOC1-C3]: عندما تستخدم معلومة من جزء معين، أدرج معرفه مباشرة بعد الجملة هكذا
هذا يتيح للنظام ربط كل ادعاء بمصدره تلقائيًا — لا تحذف هذه المعرفات
ولا تخترع معرفات غير موجودة في الوثائق المزودة.

مثال:

، بينما يذكر [DOC2-C1] تشير الدراسة إلى ارتفاع معدل القراءة بين الشباب"
[DOC4-C2] مصدر آخر أن هذا الارتفاع محدود بفئة عمرية معينة

عند مقارنة عدة وثائق (الأولوية القصوى) ##

قبل الإجابة، افحص فعليًا ما إذا كانت الوثائق المزودة تتفق أو تختلف حول
السؤال. لا تفترض الاتفاق لمجرد عدم رؤية تناقض واضح — ابحث عنه

إذا اتفقت المصادر: "تتفق المصادر على أن..." [مع كل المعرفات ذات الصلة] -
إذا اختلفت: اذكر كل موقف بوضوح مع مصدره، ثم اقترح تفسيرًا محتملاً -
للاختلاف (فترة زمنية مختلفة، منهجية مختلفة، سياق مختلف، أو خلاف حقيقي)
دون فرض تفسير واحد كحقيقة مؤكدة
إذا كان مصدر واحد فقط يتناول النقطة: نبّه القارئ صراحة أن هذا رأي -
فردى غير مدعوم بمصادر أخرى

مستوى الثقة (بدون تصنيفات صريحة) ##

"...ثقة عالية: "توضح المصادر بوضوح أن..." / "تتفق المصادر على

"...ثقة متوسطة: "يبدو أن..." / "تشير معظم المصادر إلى

"...ثقة منخفضة: "يذكر مصدر واحد أن..." / "يمكن القول بحذر أن

.لا تستخدم عبارات مثل "مستوى الثقة: عالٍ" — الثقة تُبنى داخل اللغة نفسها

عند نقص الأدلة ##

"[...]. المصادر المتاحة لا توفر إجابة كاملة. ما نعرفه هو [...]، وما ينقصنا هو"

.اقترح نوع المصدر الذي قد يسد هذه الفجوة إن أمكن

المصطلحات التقنية ##

:عند ظهور مصطلح بلا مقابل عربي مباشر

→ المصطلح الأصلي أولاً → الترجمة الصوتية بين قوسين → شرح مفاهيمي موجز

.استخدام ثابت بعد ذلك في نفس الرد

"...هي دراسة المعرفة وأصولها، وهي ما يُعنى به (Epistemology) مثال: "الإبستمولوجيا

الأسلوب ##

.اكتب كباحث زميل مطّلع يتحاور مع القارئ، لا كنظام آلي يسرد نقاطاً

اربط الأفكار بعبارات توليف طبيعية ("تتفق المصادر على..."، "هناك توجهان

.").متعارضان..."، "يبقى هذا السؤال دون إجابة حاسمة في المصادر

في نهاية الإجابة، اقترح سؤالاً منطقيًا للمتابعة إن كان ذلك مفيدًا فعليًا

.لا كصيغة ثابتة في كل رد —

اللغة ##

الرد بالكامل بالعربية الفصحى الحديثة الواضحة، بغض النظر عن لغة السؤال

أو لغة الوثائق المصدر

:الوثائق المزودة (مع المعرفات)

[DOCUMENTS_WILL_BE_INSERTED_HERE]

:سجل المحادثة

[CHAT_HISTORY_WILL_BE_INSERTED_HERE]

:سؤال المستخدم

[USER_MESSAGE_WILL_BE_INSERTED_HERE]

The load-bearing change here is the [DOC1-C3]-style inline tagging — it turns citation extraction from "parse Arabic prose and hope" into "regex for bracketed IDs," which is the difference between a feature that mostly works and one that's reliable enough to ship a 95% accuracy claim against.

4. Scope — Is 4 Weeks Realistic?

Close, but only if you cut now, not in week 3 under pressure. Realistic priority split:

Non-negotiable (the actual product):

- Upload PDF + plain text (drop Word doc parsing for launch — it's a second parsing pipeline for marginal early value)
- Chunking + embedding + per-document-diversified retrieval
- Chat with inline citations
- Arabic-only output, RTL UI

Cut or defer to week 5+:

- Word doc (.docx) upload support — real parsing complexity, low first-week value
- Freemium limits/usage tracking — launch unmetered with 5-10 trusted beta users; you don't need billing logic for a private beta
- Tags/organization — nice-to-have, zero cost to defer

- Bibliography/outline export — keep chat export (.txt) only for launch; summary/bibliography generation is a second prompt-engineering project, not a checkbox
- Auto language detection — you can let users self-tag language on upload instead of building a detector

That's roughly a week of work removed from your 4-week plan, which you should redirect entirely to **retrieval quality and citation accuracy testing** — the two things your differentiation claim actually depends on.

5. Differentiation vs. NotebookLM

Your comparison table is accurate but "Arabic-native + contradiction handling" is a positioning claim, not yet a demonstrated moat — NotebookLM can technically ingest Arabic text and Gemini itself can output Arabic reasonably well. What you're missing:

- **A concrete niche**, not "researchers" broadly. "Researchers" is NotebookLM's exact audience too. Consider owning a specific vertical where Arabic-native synthesis has outsized value and general tools underperform — e.g., Islamic studies / classical Arabic texts, MENA policy research, or Arabic literature — where source documents are themselves in Arabic and English tools mistranslate nuance rather than just translate interface labels.
 - **Academic citation format support** (APA/MLA-equivalent in Arabic academic convention) — NotebookLM doesn't target formal citation output, and this is a real unlock for grad students specifically.
 - **Visible confidence/evidence trail as a UI element**, not just prompt language — a small "evidence panel" showing which chunks fed a given answer would make your transparency principle *visible*, not just felt in the prose.
-

6. Solo Founder Gotchas at Week 3

- **Retrieval quality problems surface late and eat days.** You won't know your chunking/retrieval is weak until you test with real, messy documents in week 3 — by definition, this is discovered right when you have the least slack left. Test with real documents in week 1-2, not week 3.
- **Arabic RTL bugs compound with every UI component you add.** Tailwind/shadcn components are LTR-first; expect icon alignment, form layout, and modal positioning bugs that don't show up until you're testing on mobile in week 4, when you have zero time to fix them properly.

- **The system prompt will need real iteration, not a single write.** Budget explicit prompt-testing time (you've done this well by including test prompts in 8.6) — but that testing needs to happen *before* week 4 polish, not concurrently with it.
- **Export (.docx generation) is deceptively nontrivial** — formatting fidelity issues eat more time than expected. This is exactly why cutting it from launch scope (see #4) protects your timeline.
- **Support + bug-fixing after 5-10 beta users start using it will compete directly with any remaining build time** — budget for this explicitly rather than assuming week 4 is all launch polish.

7. Memo: v0.2 Priority Changes

Priority	Section	Change	Why
P0	5.5 RAG Pipeline	Per-document diversified retrieval, not global top-K	Contradiction detection literally cannot work otherwise
P0	6.1 System Prompt	Inline chunk-ID citation tags	Citation extraction reliability
P0	5.4 Schema	Verify actual Gemini embedding dimension before migration	Silent failure risk on day 1 of week 2
P1	4.2.1	Verify Supabase RLS is configured, not just Auth	Multi-tenant data leak risk
P1	4.1 Scope	Cut Word doc upload, freemium limits, tags from launch	Reclaim ~1 week for retrieval testing
P1	5.6 Timeline	Move real-document retrieval testing to week 1-2	Surface RAG quality problems while there's still time to fix them
P2	Section 11	Sharpen positioning to a specific research vertical	"Researchers" alone isn't differentiated from NotebookLM
P2	6.1	Add explicit token budget/truncation logic for prompt construction	Prevent silent context overflow

The single highest-leverage fix in this whole review is the retrieval diversification — everything else is polish, but that one determines whether your core differentiator

(contradiction handling) actually functions when you have real, disagreeing sources in front of the model.

Want me to draft the diversified retrieval logic (the actual query function) or turn this into a saved doc you can reference while building?

Did you say NotebookLM already works well? How much of a differentiator is this app, compared to others such as NotebookLM? What does it take to make it more appealing for Arabic users? Are you saying I should confine uploads to PDFs? Is this project worth my time?

Thought for 2s

Thought for 2s

Let me check NotebookLM's current capabilities before answering the differentiation question — that's not something I should answer from stale memory.

Reconsidered initial assumptions and prepared comprehensive answers

Reconsidered initial assumptions and prepared comprehensive answers

Good, worth checking — this actually changes my answer a bit. Let me go through each question straight.

Did I say NotebookLM already works well?

Not quite what I said — I said your "Arabic-native" claim isn't yet a *demonstrated* moat, and the search confirms why: as of 2026, NotebookLM supports over 80 languages including Arabic, and can fully parse uploaded documents in any supported language regardless of display or output language. It also generates Audio Overviews in Modern Standard Arabic and Egyptian colloquial Arabic. So "does Al-Dalil support Arabic and NotebookLM doesn't" is no longer true. That doesn't mean NotebookLM works *well* for Arabic — output language support and genuinely native-quality Arabic reasoning/rhetoric are different things — but you can't claim the feature gap anymore, only a quality gap, and quality gaps are harder to prove and market. [Google](#)

How much of a differentiator is this, realistically?

Smaller than the spec assumes, on the axis you picked. "Chat with your Arabic documents" is now something NotebookLM does out of the box. Two axes where you can still plausibly win:

1. **Depth of Arabic linguistic quality** — does Al-Dalil produce more natural, more rigorous, more *scholarly-register* Arabic synthesis than a general-purpose tool with Arabic bolted on? This is testable, not assumed — you'd need native-speaker blind comparisons against NotebookLM output on the same documents.

2. **Contradiction-handling as a structured feature**, not just prose — an explicit UI surface (evidence panel, agreement/disagreement map) that NotebookLM doesn't build toward, because it's optimizing for broad note-taking, not research synthesis. This is a product-design moat, not a language moat.

If neither of those holds up under actual side-by-side testing with real users, the honest read is that you're building a well-executed Arabic-flavored NotebookLM clone, not a differentiated product. Worth testing *before* you build four weeks of infrastructure around the assumption.

What would make it more appealing to Arabic users specifically?

Given NotebookLM already covers basic Arabic I/O, appeal has to come from things a generalist tool won't prioritize:

- Right-to-left UI that's actually native, not adapted — spacing, iconography, number formatting (Arabic-indic vs. Western numerals), date formats
- Academic Arabic citation conventions, not transliterated APA
- Handling of Arabic-specific document realities: scanned classical texts, inconsistent diacritics, dialectal variation in sources — things a MENA-focused tool would take seriously that a global tool treats as edge cases
- A genuinely narrower audience (see my earlier point about picking a vertical) where the tool's design decisions — not just its language switch — are shaped by that audience's actual workflow

Should uploads be confined to PDFs?

For *launch*, yes — that was scope-cutting advice, not a permanent architecture decision. Cutting .docx isn't about it being unimportant, it's about sequencing: PDF parsing is one pipeline, Word parsing is a second one, and building both in week 2 competes directly with the retrieval-quality work that actually determines whether your product works. Add .docx back in week 5+ once the core loop is proven.

Is this project worth your time?

I can't answer that for you — it depends on risk tolerance, opportunity cost, and how much you value building vs. the odds of a specific outcome, none of which I have visibility into. What I can tell you factually: the technical plan is buildable in 4 weeks by one competent person, and the retrieval/prompt engineering problems are real but solvable. What's *not* yet established is the differentiation — and that's worth resolving cheaply before the 4-week build, not after. A low-cost test: take 5-10 real documents from your target niche, run the same synthesis and contradiction-detection questions through both raw Gemini/NotebookLM and your v0.2 prompt, and have 2-3 native Arabic-speaking researchers blind-judge the outputs. If your version clearly wins on

Arabic quality or contradiction-surfacing, you have a real reason to build. If it's a wash, that's worth knowing before four weeks go in, not after.

Before I proceed any further, can you give me the exact prompt I should share with Google Studio AI plus whatever additional documents, based on your recommendations, with which agree? Let me test first at my level, without cost, and take it from there. I want to see a UI on Google Studio AI to studio it might land on regular users of Arabic.

Orchestrated system prompt, test documents, and Arabic UI framework for cost-free validation

Orchestrated system prompt, test documents, and Arabic UI framework for cost-free validation

Good — AI Studio in 2026 has both a **Chat mode** (system instructions field + direct file upload, completely free to test) and a **Build mode** (describe an app, get a working React UI with live preview). That maps well onto what you want: test the reasoning quality for free first, then optionally generate a UI to see how it'd feel to a real user.

Here's the exact setup.

Step 1: The system prompt (paste into AI Studio's "System instructions" field, Chat mode)

I adjusted the v0.2 prompt for this context — no chunk-ID tags, since AI Studio isn't running your RAG pipeline; it just gets full documents in context, so I switched citation to document name.

Devised multilingual test documents with intentional contradictions

Devised multilingual test documents with intentional contradictions

Step 2: Test documents (deliberate contradiction built in, so you can actually test the feature that matters)

Created 3 files, read a file

Created 3 files, read a file

How to run this test (free, no build required)

1. Go to **aistudio.google.com** → **Chat mode**.
2. Pick a Pro-tier reasoning model from the model dropdown (not the fastest/cheapest one — you're testing reasoning quality, not latency).
3. Open **System instructions**, paste the contents of al-dalil-system-prompt-aistudio.txt.

4. Upload all three test documents into the chat context.

5. Ask these questions one at a time, in Arabic:

- tests whether it —هل تتفق المصادر الثلاثة على أثر التعلم عن بعد على التحصيل الأكاديمي؟ actually notices docs 1 and 2 disagree, and doesn't just average them into mush.
- —المصدر الأول يقول إن الدرجات ارتفعت، والثاني يقول إنها انخفضت. كيف أفهم هذا التناقض؟ tests whether it explains *why* (it should surface the unmeasured internet-infrastructure confound in doc 2, which is a real methodological detail I planted).
- —ماذا تقول الدراسات عن الرفاهية النفسية للطلاب في التعلم عن بعد؟ tests whether it correctly pulls doc 3 (English source) into Arabic synthesis, and doesn't conflate wellbeing data with grade data.
- —هل هناك دليل كافٍ على أن التعلم عن بعد يحسّن الفهم الفعلي للمادة؟ tests uncertainty-acknowledgment (doc 3 explicitly says the evidence is split and shouldn't be conflated with outcomes — this is the trap question).

Judge it against your own principles: does it name sources, hold the contradiction open instead of resolving it artificially, and hedge appropriately on the last question instead of asserting a false consensus? That's your real answer on Arabic reasoning quality — independent of any UI.

Step 3: If it performs well, get a UI to gut-check with Arabic speakers

Switch to **Build mode** and paste this:

بالكامل. الواجهة تحتوي على: (RTL) ابن واجهة دردشة بسيطة باللغة العربية، اتجاه الصفحة من اليمين إلى اليسار منطقة لرفع عدة ملفات نصية، نافذة محادثة تعرض الرسائل بشكل طبيعي (المستخدم يمينًا، الرد يسارًا)، وحقل إدخال بالعربية مع زر إرسال. استخدم تصميمًا بسيطًا ونظيفًا يناسب باحثين أكاديميين، بدون أي نص إنجليزي ظاهر في الواجهة.

That gives you a working RTL prototype you can literally hand to 2-3 Arabic-speaking researchers and watch them use, at zero infrastructure cost.

One honest note: if the answers to those four test questions come back flat, generic, or fail to catch the planted contradiction — that's a real signal before you build anything, not a prompt-tuning problem to iterate past. Worth treating that outcome as informative rather than something to fix.

Al dalil system prompt aistudio

TXT

دراسة التعلم عن بعد 1 doc Test

TXT

Test doc 2 تقرير الجودة الأكاديمية 2

TXT

Test doc 3 student wellbeing survey

TXT

Help me write clear instructions for Google AI Studio about the UI for the app. The default UI is android view. This is very small and I can't act on it. Instruct Google AI studio to reshape the UI putting a laptop view as default so I can see a larger workspace. Google AI Studio produced a toggle but it does not work.

Architected responsive UI redesign replacing broken toggle with desktop-first layout system

Architected responsive UI redesign replacing broken toggle with desktop-first layout system

Looking at the screenshot — this is AI Studio rendering the preview in a fixed mobile-width frame, and it sounds like the toggle it added is just switching a CSS class that isn't actually wired to a working breakpoint. Here's a prompt that tells it to fix the actual layout logic, not just add another toggle.

Paste this into Build mode:

الواجهة الحالية مبنية بعرض ثابت يشبه شاشة الهاتف، وهذا غير مناسب لأنني أعمل من حاسوب محمول. أريد التغييرات التالية:

1. هو العرض الافتراضي عند فتح التطبيق، وليس تصميم الهاتف. لا (Desktop) اجعل تصميم سطح المكتب (CSS) للتبديل بين الاثنين — اجعل الواجهة تستخدم استعلامات الوسائط (toggle) تعتمد على زر تبديل media queries / responsive breakpoints) العرض الشاشة الفعلي
2. يحصر التطبيق داخل إطار صغير يشبه الهاتف. يجب أن (fixed max-width) أزل أي عرض أقصى ثابت (تقريباً كحد أقصى للمحتوى الرئيسي px حتى 1400) يمتد التخطيط ليملأ عرض نافذة المتصفح على الحاسوب
3. أعد ترتيب الأعمدة الثلاثة (الشريط الجانبي الأيمن للتنقل، قائمة المصادر، ومنطقة المحتوى الرئيسي) لتظهر ، بدلاً من التكديس الرأسى الحالي(px أكبر من 1024) جنباً إلى جنب بشكل أفقي على الشاشات الكبيرة
4. ، وليس (resize) تحقق من أن التبديل بين وضعي الهاتف و سطح المكتب يعمل فعلياً أثناء تغيير حجم النافذة سابق لا يعمل، احذفه تماماً واستبدله بالسلوك التلقائي toggle فقط عند التحميل الأول — إذا كان هناك زر المعتمد على حجم الشاشة
5. والتصميم البصري الحالي (الألوان، الخطوط) دون تغيير — المطلوب فقط إصلاح RTL حافظ على اتجاه وجعل سطح المكتب هو الافتراضي (layout) التخطيط

A couple of things worth checking once it applies this:

- Resize the actual preview pane wider first (drag it or maximize the browser window) — sometimes AI Studio's live preview iframe itself is capped narrow regardless of your CSS, and no prompt fixes that; you'd need to open the deployed/shared URL in a full browser tab to see true desktop width.
- If it still defaults to mobile after this, ask it explicitly: "breakpoint أخبرني ما هو الـ" — "المستخدم حالياً في الكود، ولماذا لا يتم تفعيل تصميم سطح المكتب تلقائياً" — that forces it to show you the actual CSS logic instead of patching around the symptom again.

Here is Google Studio AI response. Can you force it to show a laptop view by default?

المستخدم حالياً في الكود يعتمد التطبيق حالياً على آلية قياس مرنة (Breakpoints) الحدود القياسية. 1: الكود الحالي الذي يستعلم عن العرض المتاح BoxWithConstraints وديناميكية لتحديد نوع الشاشة وحجمها عبر مكون `maxWidth`، وينقسم كالتالي:

- يتم تفعيله تلقائياً عندما يكون العرض المتاح أكبر `(isThreeColumn)`: تصميم سطح المكتب ثلاثي الأعمدة `(1024.dp)` من أو يساوي وحدة بكسل مستقلة عن الكثافة `1024`.
- يتم تفعيله عندما يكون `(isTwoColumn)`: التصميم ثنائي الأعمدة لسطح المكتب المتوسط/الأجهزة اللوحية `dp` و `1024` العرض المتاح بين `720`.
- `dp`. يتم تفعيله عندما يكون العرض المتاح أقل من `720` `(isMobile)`: تصميم الهواتف المحمولة.
- لماذا لا يتم تفعيل تصميم سطح المكتب تلقائياً في شاشة العرض الحالية؟ يرجع السبب في ذلك إلى أبعاد المحاكى الذي يُعرض التطبيق من خلاله في المتصفح (Streaming Android Emulator) الافتراضي:
- أبعاد شاشة المحاكى: شاشة العرض الحالية تحاكي هاتفاً ذكياً بوضع عمودي قياسي، حيث يبلغ عرضها الفعلي `dp` إلى `411` في العادة ما بين `360`.
- ؛ فإن الكود يتعرف على `dp` يقع تحت حد الـ `720` (مثلاً `411`) التفاعل التلقائي: بما أن هذا العرض المتاح الجهاز تلقائياً كـ هاتف محمول ليعرض واجهة استخدام محسنة ومريحة تناسب أبعاد شاشته الصغيرة، حمايةً إذا رغبت في تجربة واجهة سطح المكتب ثلاثية الأعمدة، يمكننا لعناصر التصميم من التكس أو التداخل مؤقتاً في الكود لتعرض على المحاكى الحالي، أو يمكننا إعادة توفير (Breakpoints) تعديل حدود العرض خيار التحكم اليدوي في المظهر إذا كان ذلك يخدم أهدافك البحثية بشكل أفضل.

Show more

Acknowledged empty document content and resolved to communicate honestly

Acknowledged empty document content and resolved to communicate honestly

I genuinely cannot read it — the document content field is empty. This isn't me being cautious or hedging. There is literally no text in the <document_content> section for me to inspect or analyze.

This suggests a real problem on the upload side. Three documents in a row (20, 21, 22) have arrived with empty content, even though you're saying you can see them

displayed in Al-Dalil and they're showing substantial word counts. That's a significant disconnect.

What's most likely: When you copy the report text from Al-Dalil and paste it here into a document upload, something in that transmission chain is stripping the content — either a character encoding issue, the upload interface itself, or a handling quirk specific to how this conversation ingests documents.

Practical fix: Don't upload it. Just paste the text directly into your next message as plain text within the message itself, not as a file attachment. That bypasses the upload/document-parsing layer entirely and gets me the actual content I need to evaluate.

Given we're now deep into a very long session and you've made real progress on the app itself (the full-text extraction fix, the persistence storage, the improved glossary), this is probably a good stopping point anyway. You've got a working prototype with most of the Tier 1 features validated, and the infrastructure questions (quota, persistence, truncation) are now at least diagnosed and addressed, even if the last one needed a couple iterations.

Copy/paste the gap report text directly if you want me to evaluate it, but do that in a fresh message outside the document upload system.

Never mind. All three documents are complete. 28 Arabic concepts in working order. All five academic analysis tools are working.

This is the best screenshot of the whole session — and genuinely the right place to pause and take stock.

What image 1 confirms

Three documents with real, substantial word counts: 5,128 / 6,652 / 16,139 words. That's not truncation. The extraction fix actually worked, and the numbers are exactly what you'd expect from full academic articles, not the 75-117 word fragments from earlier. The glossary has grown to 26 terms (المصطلحات والمفاهيم 26), all reportedly in correct Arabic — a significant jump from the 3 that survived the data-loss incident.

What image 2 confirms

All five Synthesis Editor tools are present and selectable: توليف عام ومقارنة, مصفوفة الأدلة, تقرير الفجوات, التوصيات والآثار, الأسئلة الشائعة. The report visible in the preview pane shows the scope disclosure working ("يعتمد على مصدر واحد نشط من بين 1"), the three-section structure

intact, and the quote-then-claim format visible in the prose. The "حفظ في السجل" (Save to History) and "نسخ التقرير" (Copy Report) buttons are present.

What this session accomplished, in total

Looking back across the full arc: you went from a prototype that was silently fabricating statistics, silently swallowing quota errors, silently corrupting short documents, silently substituting cached placeholder text for real AI output, and phonetically transliterating half its glossary — to a system that now surfaces all of those failures visibly, quotes source text verbatim before claiming anything, stores data persistently across rebuilds, and produces correct Arabic academic terminology across 26 concepts from three different documents.

That's a lot of ground covered in one session.

What's still genuinely open, for next time

Three things worth picking up fresh rather than tonight:

The **cross-domain generalization test** — everything validated so far is still education-domain content. One document from an unrelated field through the glossary and gap report would close that question definitively.

The "توليف عام ومقارنة" vs "التوصيات والآثار" **overlap question** — once you have more sources active, worth generating both on the same question and checking whether they're genuinely distinct or largely redundant now that the policy-report fix removed the matrix duplication.

The **agreement meter** — still the one Tier 1 feature never confirmed working in a chat screenshot. Worth a single chat question that forces cross-source comparison and checking whether the meter actually renders underneath it.

Good session. Rest well.